SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO2 - Manipulation (BL3, BL4)	**Assessment:**	Assessment Tool 1 - Analytical Problem Solving
Weightage:	30% of CIA	**Total Marks:**	100

CO2 Statement: *Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.*

Student Name:	B.Sathvika	**Reg. No.:**	192525224
Section / Batch:	AIML & 2nd year	**Date:**	17/07/2026

Code of Conduct

I B. sathvika (192525224) *(Name / Reg No)* *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Sathvika

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
	GRAND TOTAL:		**100 Marks**

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge

Signature: Ungrey

Date: __________

Course Coordinator

Signature: __________

Date: __________

Program Director

Signature: __________

Date: __________

Name: B. Sathvika
RegNo: 192525224

CO_2 - AT_1 Analytical Problem

1) **Doubly linked list:**

Initial list

Assume the six elements are:

$A \rightleftarrows B \rightleftarrows C \rightleftarrows D \rightleftarrows E \rightleftarrows F$

Insert X at 3rd position

The new elements X is inserted between B and C.

$A \rightleftarrows B \rightleftarrows C \rightleftarrows D \rightleftarrows E \rightleftarrows F$

Change the links:

X. prev = B
X. next = C
B. next = X
C. prev = X

After insertion:

$A \rightleftarrows B \rightleftarrows X \rightleftarrows C \rightleftarrows D \rightleftarrows E \rightleftarrows F$

Now X is at position 3.

Step 2: Delete the element at 2nd position

The element at position 2 is B.

Before deletion:

$A \rightleftarrows B \rightleftarrows X \rightleftarrows C \rightleftarrows D \rightleftarrows E \rightleftarrows F$

change the links:

A. next = X
X. prev = A

Delete node B.

Final list:

A ⇄ X ⇄ C ⇄ D ⇄ E ⇄ F

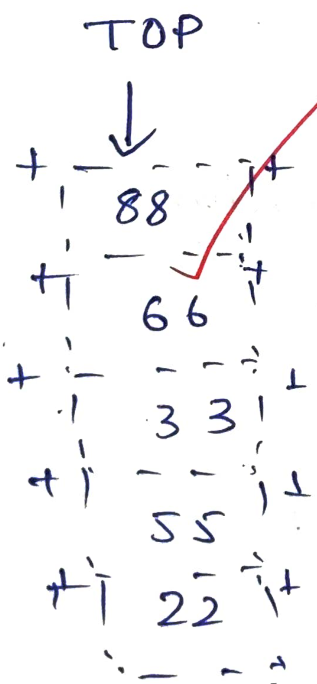
Analysis:

- Traversal : O(n)
- Insertion after locating position: O(1)
- Deletion after locating position: O(1)
- Extra space : O(1)

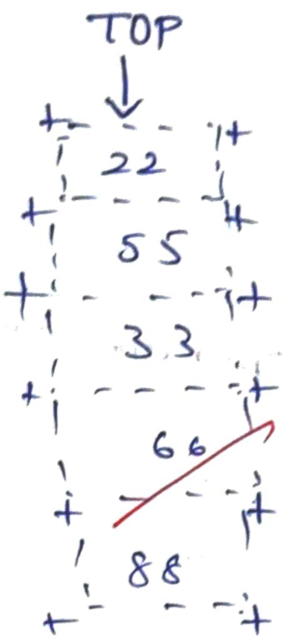
2. Stack operations:

A stack follows LIFO (last in, first out).

Initial stack:



1. Invert the stack:



2. pop( )

Remove 22.

55
33
66
88

3. pop( )

Remove 55.

33
66
88

4. pop( )

Remove 33.

66
88

5. push(90)

90 ← TOP
66
88

6. push(36)

36
90
66
88

7. push(11)

11
36
90
66
88

The stack is now full.

8. push(88)

Since the stack size is only 5, this operation causes:

STACK OVERFLOW:

the new 88 is not inserted.

~~11~~ → TOP
~~36~~
90
66
88

9. pop()

Remove 11.

36 → TOP
90
66
88

10. pop()
Remove 26.

90 → TOP
66
88

Final Answer:

TOP
↓

90
66
88

final TOP = 90